

GALAXEYE SPACE

Technical Report — AI Research Intern

Building Damage Change Detection Using U Net on EO SAR Image Pairs

Field	Details
Task	Binary Pixel Level Change Detection
Model	U Net (Encoder Decoder with Skip Connections)
Input	6 Channel Concatenated Pre + Post Images
Training Platform	Google Colab - Tesla T4 GPU
Framework	PyTorch
Final F1 Score	0.7664
Final IoU	0.6213

Title : Building Damage Detection using Satellite Images

2 . OBJECTIVE :

The objective of this project was to develop a deep learning based change detection system capable of identifying damaged building regions from pre event and post event satellite imagery. A U Net semantic segmentation model was implemented to perform binary pixel level classification of changed and unchanged regions. The system processes paired Electro Optical (EO) and Synthetic Aperture Radar (SAR) images captured before and after a disaster event, producing a binary change mask where each pixel is labelled as either Changed (damaged) or Unchanged.

This task has direct real world applications in disaster response and recovery enabling rapid automated assessment of building damage across large geographic areas from satellite observations, reducing the time and human effort required compared to manual inspection.

3. Dataset Description

The dataset was provided by GalaxEye Space and consists of co registered pre event and post event remote sensing image pairs with corresponding pixel level annotation masks across multiple disaster events. The dataset was provided with a fixed train/validation/test split, which was used without modification.

3.1 Dataset Structure

- Pre event images: Satellite imagery captured before the disaster event, stored as multi band TIF files (e.g., scene_01_000001_building_damage.tif).
- Post event images: Satellite imagery captured after the disaster event, with identical filenames to pre event images for direct pairing.
- Target masks: Pixel level annotation masks containing original semantic class labels (0–3).
- Training set: 2,781 image pairs covering multiple geographic regions and disaster types.

3.2 Label Remapping

The original annotations contained four semantic classes. Before any training or evaluation, labels were remapped to binary as follows:

Original Class	Original Value	Remapped Value	Remapped Class
Background	0	0	No Change
Intact	1	0	No Change
Damaged	2	1	Change
Destroyed	3	1	Change

This remapping was applied consistently across all splits before model training and metric computation. The resulting binary segmentation labels represent 0 = Unchanged pixels and 1 = Changed/Damaged pixels.

4.Preprocessing

All images were preprocessed through a consistent pipeline before being fed to the model. The following steps were applied to every pre event and post event image pair:

- **Resizing to 256×256:** All input images were resized to a fixed 256×256 pixel resolution using bilinear interpolation to ensure uniform input dimensions across the dataset.
- **Mask resizing :** Corresponding target masks were resized to 256×256 using nearest neighbor interpolation to preserve discrete integer label values without introducing interpolated intermediate values.
- **Tensor conversion :** Images were converted from NumPy arrays to PyTorch float tensors and normalised to the [0, 1] range by dividing by 255.
- **6 channel concatenation :** Pre event and post event images (each 3 channels — RGB) were concatenated along the channel dimension to produce a single 6 channel input tensor of shape (6, 256, 256). This early fusion approach allows the model to learn temporal differences directly from the combined input.
- **Label binarization :** Target mask values were remapped from the original 4 class labels (0–3) to binary labels (0 and 1) as described in Section 3.2, before conversion to a PyTorch long tensor.

5. Model Architecture

The U Net architecture (Ronneberger et al., 2015) was selected for its proven effectiveness in pixel level segmentation tasks and its ability to produce high resolution outputs while preserving spatial detail through skip connections.

5.1 Encoder (Contracting Path)

The encoder progressively reduces spatial resolution while increasing feature channel depth. Each encoder block consists of two 3×3 convolutional layers followed by batch normalisation and ReLU activation. Max pooling with stride 2 is applied between blocks to downsample the feature maps. The encoder extracts hierarchical features capturing edges, textures, and high level semantic structures from the 6 channel input.

5.2 Bottleneck

The bottleneck is the deepest layer of the network, operating at the lowest spatial resolution. It captures the most abstract, semantically rich features from the input image pair before the upsampling path begins.

5.3 Decoder (Expanding Path)

The decoder progressively restores spatial resolution through transposed convolutions (upsampling). At each decoder stage, the upsampled feature map is concatenated with the corresponding encoder feature map via skip connections. This mechanism preserves fine grained spatial detail that would otherwise be lost during downsampling, enabling accurate pixel level boundary localisation of damaged regions.

5.4 Output Head

The final layer is a 1×1 convolution that maps the feature channels to 2 output channels (one per class: No Change and Change). The output logits are passed to CrossEntropyLoss during training and converted to class predictions via argmax at inference.

Component	Description
Input	6 channel tensor (3ch pre event + 3ch post event), 256×256
Encoder	4 downsampling blocks — Conv → BN → ReLU → MaxPool
Bottleneck	Deepest feature representation
Skip Connections	Feature maps from each encoder block concatenated to decoder
Decoder	4 upsampling blocks — TransposeConv → Concat → Conv → BN → ReLU
Output	1×1 Conv → 2 channels (No Change, Change)

6. Training Configuration

The following hyperparameters were used for model training:

Parameter	Value
Model	U Net
Epochs	20
Optimizer	Adam
Loss Function	CrossEntropyLoss
Batch Size	1
Image Size	256 × 256
Input Channels	6 (pre + post concatenated)
Output Classes	2 (No Change, Change)
Platform	Google Colab
GPU	Tesla T4 (16GB VRAM)
Framework	PyTorch

A batch size of 1 was used due to GPU memory constraints on Google Colab. Cross Entropy Loss was selected as the loss function for its direct applicability to multi class segmentation. The Adam optimiser was chosen for its adaptive learning rate properties and stability on segmentation tasks.

Results

7.1 Quantitative Metrics

All metrics were computed for the Change class (label = 1):

Metric	Value
IoU	0.6213
Precision	0.7684
Recall	0.7645
F1 Score	0.7664

7.2 Confusion Matrix

The confusion matrix below summarises pixel level classification results:

	Predicted: No Change (0)	Predicted: Change (1)
Actual: No Change (0)	64,308 (TN)	230 (FP)
Actual: Change (1)	235 (FN)	763 (TP)

7.3 Analysis :

The model achieved an F1 Score of 0.7664 and IoU of 0.6213, representing a strong baseline for binary change detection on this dataset. The closely matched Precision (0.7684) and Recall (0.7645) indicate a well balanced model it neither aggressively over predicts damage (low precision) nor systematically misses damaged regions (low recall).

The confusion matrix reveals that the model correctly identified 763 changed pixels (True Positives) while producing 230 false positives and 235 false negatives — a very balanced error profile. The 64,308 true negatives confirm that the model handles the dominant no change class correctly. False negatives (missed damage) are slightly more frequent than false positives, which is consistent with the class imbalance in the dataset where change pixels are rare.

Challenges Faced :

Several practical challenges were encountered during the implementation and training phases of this project:

- **CUDA runtime crashes:** Google Colab's Tesla T4 GPU sessions disconnected multiple times during training, requiring restarts from the last saved checkpoint. This limited the number of complete training runs and hyperparameter experiments that could be conducted.
- **GPU session time limits:** Colab's free tier imposes a maximum session duration. Training had to be structured carefully to complete within a single session, which restricted the number of epochs and prevented extensive experimentation with different architectures.
- **Memory issues:** With high resolution satellite imagery and a U Net architecture, GPU memory was a limiting factor. Batch size had to be set to 1 to avoid out of memory errors, which slowed training and increased gradient noise.
- **Dataset extraction problems:** The dataset was distributed as multiple compressed ZIP archives. Extraction on Windows and uploading to cloud platforms required careful management of folder structures and path configurations to match the expected layout.
- **Class imbalance:** The dataset exhibited significant class imbalance with changed pixels representing a small fraction of total pixels. This required careful loss function design and weight tuning to prevent the model from predicting no change for all pixels.

Future Improvements :

Due to computational limitations and Google Colab GPU runtime restrictions, the current implementation primarily focused on training and evaluating the model on the training dataset to establish a working baseline segmentation pipeline. Frequent runtime disconnections, memory constraints, and limited GPU availability restricted extensive experimentation using separate validation and test splits within the available project timeline

If additional uninterrupted GPU runtime and computational resources had been available, the next steps would have included creating dedicated validation and test datasets, performing hyperparameter tuning, applying advanced data augmentation techniques, experimenting with deeper architectures such as Attention U Net and Transformer based models, and conducting extensive performance benchmarking to improve model generalisation and robustness. However, the implemented framework is fully extensible, and future work will include comprehensive validation/testing, hyperparameter tuning, and cross dataset evaluation to further enhance segmentation accuracy and reliability.

Specific planned improvements include:

- **Attention U Net:** Incorporate attention gates in the skip connections to allow the model to focus on relevant spatial regions and suppress irrelevant background activations. This is particularly useful for detecting small, scattered damaged structures.
- **Transformer architectures:** Explore Change Former and BIT CD, which use self attention to model long range temporal dependencies between pre and post image features — the current state of the art in change detection.

- **Dedicated validation and test splits:** Properly evaluate generalisation on held out data separate from training to detect overfitting and obtain unbiased performance estimates.
- **Data augmentation:** Apply systematic augmentation including horizontal/vertical flips, random rotations, brightness/contrast variation, and Gaussian noise to improve model robustness and reduce overfitting on the relatively small training set.
- **Hyperparameter tuning:** Systematically explore learning rates, batch sizes, loss function combinations (Dice+BCE, Focal loss), and decision thresholds to further improve F1 and IoU scores.
- **Multi GPU or Kaggle P100:** Use Kaggle's free P100 GPU with longer session limits to run larger experiments without runtime interruption.

10. Conclusion

This project successfully demonstrated an end to end deep learning pipeline for building damage change detection using remote sensing imagery. The implemented U Net model achieved strong baseline segmentation performance despite limited computational resources and runtime constraints, reaching an F1 Score of 0.7664 and IoU of 0.6213 on the change class.

The key design decisions 6 channel early fusion of pre/post image pairs, label remapping from four semantic classes to binary change masks, and Cross Entropy Loss produced a well balanced model with nearly equal precision and recall. The confusion matrix confirms that the model correctly handles the dominant no change class (64,308 TN) while successfully detecting 763 changed pixels with only 230 false positives and 235 false negatives.

The architecture and training pipeline are fully modular and reproducible, providing a solid foundation for future improvements including attention mechanisms, transformer based architectures, comprehensive validation/test evaluation, and advanced augmentation strategies. The project demonstrates that even with a classical U Net and standard training configuration, meaningful change detection performance is achievable on real world disaster satellite imagery.

11. References

1. Ronneberger, O., Fischer, P., & Brox, T. (2015). U Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015. <https://arxiv.org/abs/1505.04597>
2. Long, J., Shelhamer, E., & Darrell, T. (2015). Fully Convolutional Networks for Semantic Segmentation (FCN). CVPR 2015. <https://arxiv.org/abs/1411.4038>
3. Oktay, O., et al. (2018). Attention U Net: Learning Where to Look for the Pancreas. MIDL 2018. <https://arxiv.org/abs/1804.03999>
4. Bandara, W. G. C., & Patel, V. M. (2022). A Transformer Based Siamese Network for Change Detection. IGARSS 2022. <https://arxiv.org/abs/2201.01293>
5. Chen, H., et al. (2021). Remote Sensing Image Change Detection with Transformers (BIT CD). IEEE TGRS. <https://arxiv.org/abs/2103.00208>
6. Daudt, R. C., et al. (2018). Fully Convolutional Siamese Networks for Change Detection. ICIP 2018. <https://arxiv.org/abs/1810.08462>
7. Fang, S., et al. (2021). SNUNet CD: A Densely Connected Siamese Network for Change Detection. IEEE GRSL.
8. Lin, T. Y., et al. (2017). Focal Loss for Dense Object Detection. ICCV 2017. <https://arxiv.org/abs/1708.02002>
9. segmentation models pytorch library: https://github.com/qubvel/segmentation_models.pytorch
10. GalaxEye Space Technical Assignment Dataset (2026) — provided dataset.

PERSONAL NOTE :

I would like to sincerely mention that this project was completed with full honesty and genuine effort. I already possess strong foundational knowledge in Machine Learning, Deep Learning, NumPy, Pandas, and data analysis workflows. However, PyTorch was relatively new to me, and this assignment became an opportunity to rapidly learn and implement a complete deep learning pipeline under real-world constraints.

At the same time, this period also coincides with the final phase of my college academic schedule, project submissions, and placement activities.

Despite these constraints, I was highly motivated to work on this assignment because the problem statement itself was genuinely exciting to me. My interests extend beyond academics into areas such as geopolitics, satellite intelligence, remote sensing applications, and global strategic technologies, which made GalaxEye's work particularly inspiring.

Interestingly, around the same time this internship opportunity was announced, I also came across your organization's growing presence across platforms like YouTube and X (Twitter), which further increased my curiosity and enthusiasm toward the company's mission and technical direction.

If uninterrupted GPU access and stronger computational resources such as Colab Pro or dedicated hardware had been available, I would have further expanded this work using advanced architectures, larger experimentation pipelines, validation/testing frameworks, and more extensive benchmarking.

Regardless of the internship outcome, I genuinely appreciate the opportunity to work on this assignment and learn from it. I would be extremely excited to contribute, learn, and work with the team in the future. Even if I am not selected for this internship cycle, I would still be grateful if the team considers me for any future opportunities, collaborations, or technical work where my skills and enthusiasm may be valuable.

Thank you for reviewing my submission.